SHOFIA FAUZIYAH

Course by NordEdu
SQL #7

15 DAYS OF USING SQL

Mastering Data Retrieval & Management with PostgreSQL

Challenge

SYNTAX

```sql
-- Show only those movie titles, their associated film_id and replacement_cost
-- with the lowest replacement_costs for in each rating category - also show the
-- rating.
SELECT
    f.title,
    f.film_id,
    f.replacement_cost,
    f.rating
FROM film f
WHERE f.film_id = (
    SELECT f2.film_id
    FROM film f2
    WHERE f2.rating = f.rating
    ORDER BY f2.replacement_cost ASC, f2.film_id ASC
    LIMIT 1
)
ORDER BY f.film_id ASC;
```

OUTPUT

Data Output | Messages | Notifications

Showing rows: 1 to 5

	title text	film_id [PK] integer	replacement_cost numeric (5,2)	rating mpaa_rating
1	ANACONDA CONFESSIO...	23	9.99	R
2	CIDER DESIRE	150	9.99	PG
3	CONTROL ANTHEM	182	9.99	G

Total rows: 5 | Query complete 00:00:04.590

notes

Menggunakan subquery untuk memilih atau mem filter film dengan replacement_cost paling rendah pada setiap kategori rating. Jika terdapat beberapa film dengan biaya penggantian yang sama, maka film dengan film_id terkecil yang akan dipilih. Hasil ditampilkan dan diurutkan berdasarkan film_id (ASC).

Challenge

SYNTAX

```sql
-- Show only those movie titles, their associated film_id and the length that have
-- the highest length in each rating category - also show the rating.
SELECT
    f.title,
    f.film_id,
    f.rating,
    f.length
FROM film f
JOIN (
    SELECT rating, MAX(length) AS max_length
    FROM film
    GROUP BY rating
) AS sub
ON f.rating = sub.rating
AND f.length = sub.max_length
ORDER BY f.film_id ASC;
```

OUTPUT

Data Output | Messages | Notifications

Showing row

	title text	film_id [PK] integer	rating mpaa_rating	length smallint
1	CHICAGO NORTH	141	PG-13	185
2	CONTROL ANTHEM	182	G	185
3	CRYSTAL BREAKING	198	NC-17	184

Total rows: 14 | Query complete 00:00:00.231

notes

Mencari (length) paling tinggi pada setiap kategori rating, kemudian hasil subquery digabungkan (JOIN) dengan tabel film berdasarkan rating dan length yang sama.

Hanya film yang memiliki durasi terpanjang pada setiap kategori rating yang akan ditampilkan.

Hasil akhirnya diurutkan berdasarkan film_id secara (ASC)

Challenge

SYNTAX

```sql
-- Show all the payments plus the total amount for every customer as well as the
-- number of payments of each customer.
SELECT
    p.payment_id,
    c.customer_id,
    p.staff_id,
    p.amount,
    agg.sum_amount,
    agg.count_payments
FROM customer c
INNER JOIN payment p
    ON c.customer_id = p.customer_id
INNER JOIN (
    SELECT customer_id,
           SUM(amount) AS sum_amount,
           COUNT(payment_id) AS count_payments
    FROM payment
    GROUP BY customer_id
) agg
    ON c.customer_id = agg.customer_id
ORDER BY c.customer_id,p.amount DESC, p.payment_date;
```

OUTPUT

Data Output | Messages | Notifications

SQL | Showing rows: 1 to 1000 | Page No

	payment_id integer	customer_id integer	staff_id smallint	amount numeric (5,2)	sum_amount numeric	count_payments bigint
1	18497	1	2	9.99	118.68	32
2	28997	1	1	7.99	118.68	32
3	18495	1	1	5.99	118.68	32

Total rows: 16049 | Query complete 00:00:00.396

notes

Membuat relasi tabel customer dengan payment untuk menampilkan data pembayaran setiap pelanggan. Selanjutnya, menghitung total pembayaran dan jumlah transaksi setiap pelanggan berdasarkan customer_id.
Hasil diurutkan berdasarkan customer_id, kemudian jumlah pembayaran dari yang terbesar, dan jika sama akan diurutkan berdasarkan tanggal pembayaran.

Challenge

SYNTAX

```sql
-- Show only those films with the highest replacement costs in their rating
-- category plus show the average replacement cost in their rating category.
SELECT
    f.title,
    f.replacement_cost,
    f.rating,
    avg_table.avg_replacement_cost
FROM film f
INNER JOIN (
    SELECT
        rating,
        AVG(replacement_cost) AS avg_replacement_cost,
        MAX(replacement_cost) AS max_replacement_cost
    FROM film
    GROUP BY rating
) avg_table
    ON f.rating = avg_table.rating
WHERE f.replacement_cost = avg_table.max_replacement_cost
ORDER BY f.title ASC;
```

OUTPUT

Data Output | Messages | Notifications

Showing rows: 1 to 53

	title text	replacement_cost numeric (5,2)	rating mpaa_rating	avg_replacement_cost numeric
1	ARABIA DOGMA	29.99	NC-17	20.1376190476190476
2	BALLROOM MOCKINGBIRD	29.99	G	20.1248314606741573
3	BLINDNESS GUN	29.99	PG-13	20.4025560538116592

Total rows: 53 | Query complete 00:00:00.245

notes

Film yang memiliki (replacement_cost) paling tinggi di setiap kategori rating. Juga menampilkan rata-rata biaya penggantian untuk kategori rating tersebut agar bisa dibandingkan. Menghitung rata-rata (AVG) dan nilai tertinggi (MAX) dari replacement_cost berdasarkan rating, lalu hasilnya digabungkan dengan tabel film.
Hanya film yang memiliki replacement_cost tertinggi pada masing-masing rating yang ditampilkan.

Challenge For Today
#DAY7

Challenge

SYNTAX

```sql
-- Show only those payments with the highest payment for each customer's first
-- name - including the payment_id of that payment.
-- How would you solve it if you would not need to see the payment_id?
SELECT
    c.first_name,
    p.amount,
    p.payment_id
FROM payment p
INNER JOIN customer c
    ON p.customer_id = c.customer_id
WHERE p.amount = (
    SELECT MAX(p2.amount)
    FROM payment p2
    INNER JOIN customer c2
        ON p2.customer_id = c2.customer_id
    WHERE c2.first_name = c.first_name
)
ORDER BY p.payment_id ASC;
```

OUTPUT

Data Output | Messages | Notifications

	first_name text	amount numeric (5,2)	payment_id integer
1	PENNY	8.99	16058
2	BRANDY	10.99	16073
3	DAVID	10.99	16125

Total rows: 840 | Query complete 00:00:00.522

notes

Menampilkan pembayaran dengan nominal paling besar untuk setiap pelanggan yang memiliki (first_name) yang sama, menampilkan payment_id dari pembayaran tersebut.

Join tabel payment dengan customer, menggunakan subquery untuk mencari jumlah pembayaran (MAX(amount)) terbesar berdasarkan first_name.

Result data pembayaran yang memiliki nominal terbesar untuk setiap nama depan yang ditampilkan, diurutkan berdasarkan payment_id

THANKYOU